Chapter 1 – Introduction

1.1 Overview

In today's digital era, educational institutions handle large volumes of student data on a daily basis. Managing this data manually — through registers, spreadsheets, or disconnected files — is time-consuming, error-prone, and difficult to scale. There is a growing need for a structured, automated system that can reliably store, retrieve, update, and manage student records.

The Student Management System (SMS) is a software application designed to address this need. It provides a complete digital solution for student record management, developed in two complementary platforms:

- Core Java Console Application – A terminal-based program demonstrating Object-Oriented Programming (OOP) principles with a clean 3-layer architecture.
- Web-Based Interface – A modern HTML5/CSS3/JavaScript single-page application featuring a live dashboard with visual analytics, real-time search, and browser-based data persistence using localStorage.

Together, these two versions demonstrate a complete understanding of software development — from foundational Java programming to a deployable web interface — making this project both academically rigorous and practically relevant.

1.2 Objectives

The primary objectives of the Student Management System are:

1. To develop a reliable, user-friendly platform for managing student records digitally.
2. To demonstrate core OOP concepts: encapsulation, constructors, getters/setters, toString() override, and Optional.
3. To implement a clean 3-layer software architecture (Model → Repository → UI) for maintainability.
4. To eliminate manual record-keeping and reduce data errors through input validation and duplicate detection.
5. To provide fast record access through partial, case-insensitive name search.
6. To create an interactive web dashboard with real-time bar chart and donut chart analytics.
7. To persist data across browser sessions using localStorage in the web version.
8. To build a scalable foundation that can be extended with Spring Boot and MySQL for production use.

1.3 Project Category & Scope

The Student Management System is classified as an Academic Record Management Application. It is a dual-platform project spanning both console and web technologies.

Parameter	Details
Application Type	Console CRUD App + Web Dashboard App
Domain	Educational Technology / Academic Administration
Platform	Java Console (Terminal) + Web Browser (HTML/JS)
Architecture	3-Layer (Model – Repository – UI)
Data Storage	In-memory ArrayList (Java) + localStorage (Web)
Scalability	Extendable to Spring Boot REST API + MySQL

Chapter 2 – System Requirements

2.1 Software Requirements

Component	Specification
Operating System	Windows 10 / Windows 11
Console Language	Core Java — JDK 17 or above
Web Languages	HTML5, CSS3, JavaScript (ES6)
IDE / Code Editor	VS Code / IntelliJ IDEA
Web Browser	Google Chrome / Mozilla Firefox
Compiler	javac (bundled with JDK)

2.2 Hardware Requirements

Components	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum (8 GB recommended)
Storage	500 GB Hard Disk or more
Display	1024 × 768 resolution or higher
Input Devices	Keyboard and Mouse

Chapter 3 – System Architecture

The system follows a clean 3-Layer Architecture that separates responsibilities clearly. Each layer communicates only with its adjacent layer, making the code maintainable and extensible.

Layer	File	Responsibility
Model	Student.java	Holds student data, auto-ID generation, encapsulation, toString()
Repository	StudentRepository.java	Manages ArrayList, CRUD operations, search, and Optional handling
UI / Main	StudentManagement.java	Menu loop, input reading, validation logic, formatted display

The web version follows the same conceptual separation: the HTML form acts as the UI layer, JavaScript functions act as the repository layer (CRUD on localStorage), and the student data object acts as the model.

Chapter 4 – Data Design

4.1 Entity-Relationship (ER) Diagram

The ER Diagram represents the data entities involved in the Student Management System and the relationships between them.

ER DIAGRAM — Student Management System

Attribute	Details
student_id (PK)	Integer — Auto-generated, unique
name	String — Full student name
age	Integer — Valid range: 16–60
course	String — Enrolled course name
email	String — Optional, student email
addedOn	Date — Record creation date

Relationship Description:

- One STUDENT record has exactly one set of attributes (student_id, name, age, course, email, addedOn).
- The COURSE is stored as a string attribute within the Student entity (no separate Course table in the current version).
- In the web version, STUDENT records are stored as JSON objects in the browser's localStorage under the key 'sms_students'.
- In the Java version, STUDENT objects are held in an in-memory ArrayList managed by the StudentRepository class.

Figure 4.1 – Entity-Relationship Diagram

4.2 Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD shows the system as a single process and its interaction with external entities. It provides a high-level overview of data entering and leaving the system.

Input Data Flows (User → System)	Output Data Flows (System → User)
Student name, age, course, email	Confirmation messages (success/error)
Search keyword (name or course)	Filtered student records list
Student ID for update/delete	Updated records / deletion status
Menu selection / form submission	Dashboard analytics (charts, stats)

Figure 4.2 – DFD Level 0 (Context Diagram)

4.3 Data Flow Diagram – Level 1

The Level 1 DFD expands the single process into its main sub-processes, showing how data flows between each module of the system.

P#	Process Name	Input	Output	Data Store
P1	Add Student	Name, Age, Course, Email	Success / Error message	localStorage / ArrayList
P2	View Students	Request to view all	Full student list	localStorage / ArrayList
P3	Search Student	Partial name or course	Matching records	localStorage / ArrayList
P4	Update Student	Student ID + new values	Updated record	localStorage / ArrayList
P5	Delete Student	Student ID + confirmation	Record removed	localStorage / ArrayList
P6	Dashboard Analytics	All student records	Charts + stat cards	localStorage (read)
P7	Validate Input	Raw user input	Valid data or error	N/A (in-process)

Figure 4.3 – DFD Level 1 (Process Decomposition)

Chapter 5 – Program Flowchart

The following flowchart describes the overall execution flow of the Java Console Application, from program start to exit.

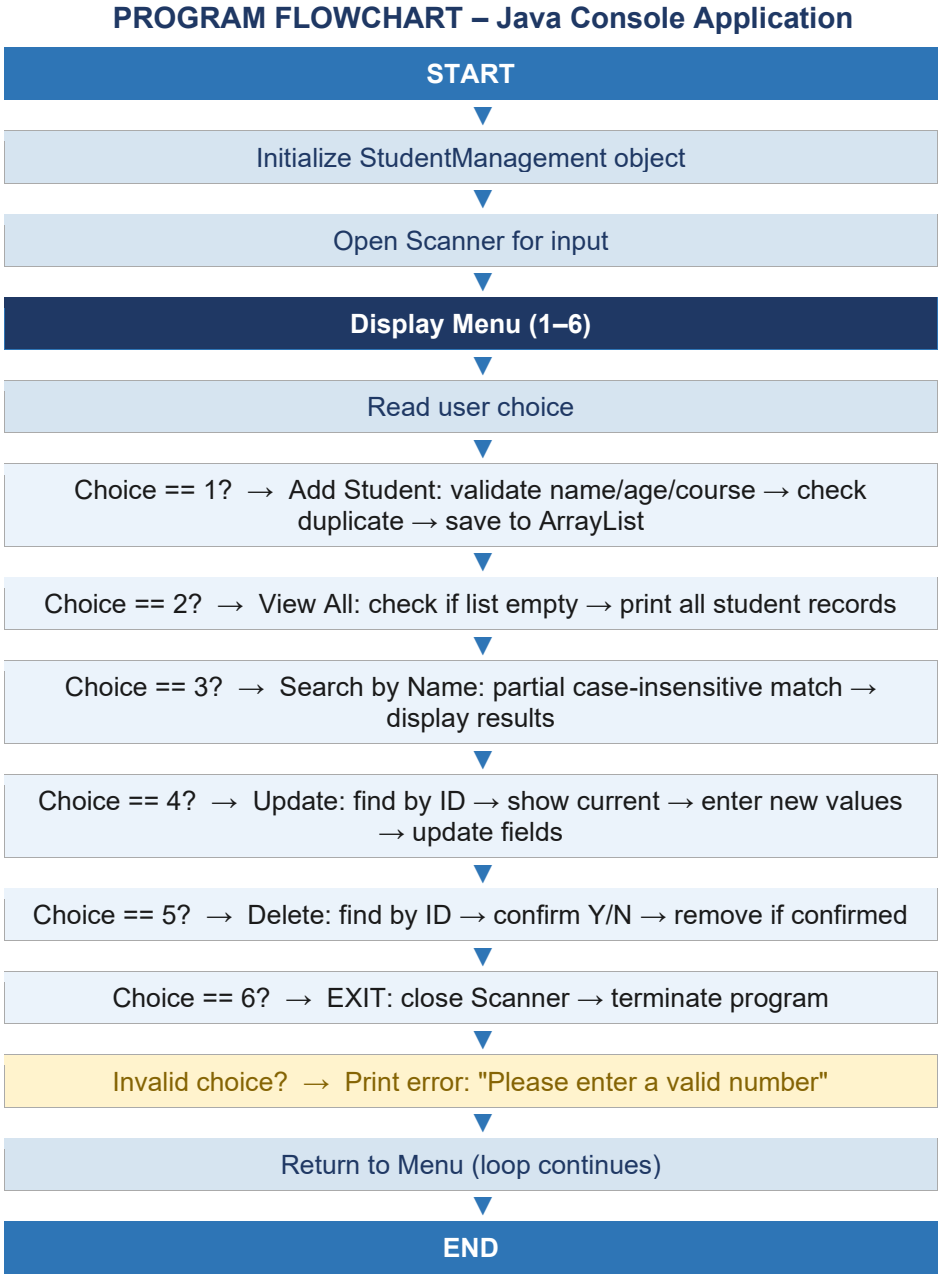

Figure 5.1 – Program Flowchart (Java Console Application)

5.1 Web Application Flow

The web application follows a similar flow but operates through a browser interface with three navigable pages:

- User opens the web app → Dashboard loads with stat cards and charts.
- User navigates to 'Add Student' → Fills form → Clicks 'Add Student' → Data validated → Saved to localStorage → Dashboard updates automatically.
- User navigates to 'All Students' → Table displayed → Can search, edit, or delete any record in real time.
- Page refresh → Data loaded from localStorage → No data loss.

Chapter 6 – Module Description

6.1 Java Console Application Modules

6.1.1 Student.java — Model Class

The Student class is the core data model. It uses private fields with a static idCounter for auto-ID generation. Public getters and setters enforce encapsulation, and the toString() method is overridden to display formatted output automatically.

Field	Type	Description
id	int (static counter)	Auto-generated unique identifier
name	String	Full name of the student
age	int	Student age (16–60 valid range)
course	String	Enrolled course name

6.1.2 StudentRepository.java — Data Layer

Manages the ArrayList with CRUD methods. Using Optional<Student> prevents NullPointerException on lookup failures. Key methods include:

- add(Student s) – Appends a validated student to the ArrayList.
- getAll() – Returns the full list of students.
- findById(int id) – Returns Optional<Student> to safely handle missing records.
- findByName(String keyword) – Partial, case-insensitive search returning all matches.
- deleteById(int id) – Removes student after Optional lookup.
- isDuplicate(String name, String course) – Checks for existing records before adding.

6.1.3 StudentManagement.java — UI Layer

Uses instance-based design — main() creates a new StudentManagement object and calls run(). A do-while loop ensures the menu appears at least once. Centralised readInt() and readString() helper methods eliminate repeated try-catch blocks.

Option	Feature	Description
1	Add Student	Reads name, age, course — validates all fields, checks duplicates, saves.
2	View All Students	Displays all records in a numbered list with formatted output.
3	Search by Name	Partial, case-insensitive search returning all matching records.
4	Update Student	Finds by ID, shows current values, updates only changed fields.
5	Delete Student	Finds by ID, shows record, asks yes/no confirmation, removes.
6	Exit	Closes Scanner and exits the program cleanly.

6.2 Web Application Modules

The web application is a single HTML5 file with three navigable pages connected via a sidebar. All data is managed through JavaScript with localStorage as the persistence layer.

Page / Module	Features
Dashboard	Stat cards (total students, avg age, unique courses, youngest), bar chart (students per course), donut chart (% distribution), recently added table.
All Students	Live search by name or course, full data table with Edit/Delete actions per row, record count display, avatar initials.
Add Student	Registration form with validation (name, age 16–60, course dropdown, optional email), saves to localStorage, updates dashboard in real time.
localStorage	JSON array stored under key 'sms_students'. Persists across page refreshes and browser restarts. Each record: {id, name, age, course, email, addedOn}.

Chapter 7 – Project Screenshots

This chapter presents the actual screenshots from the working Student Management System, demonstrating both the web interface and the Java console application.

7.1 Web Application – Dashboard

The Dashboard displays a real-time overview of all student records. The stat cards at the top show total students, average age, number of unique courses, and the youngest student's age. The bar chart visualises students per course, and the donut chart shows percentage distribution. The recently added table lists the four most recently registered students.

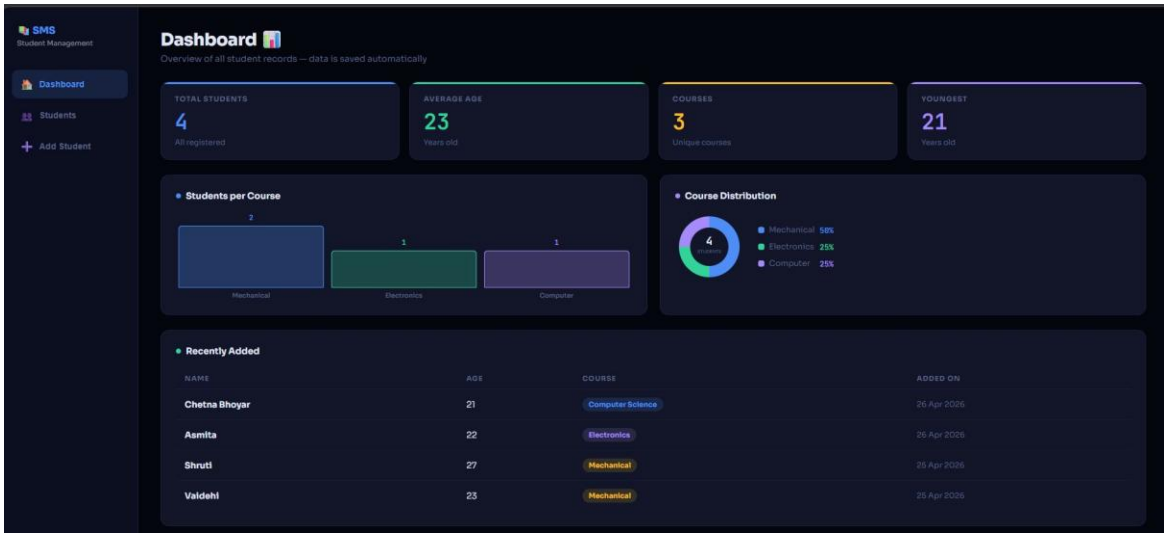

Figure 7.1 – Dashboard Page: Stat Cards, Bar Chart, Donut Chart, Recently Added Table

7.2 Web Application – All Students Page

The All Students page lists every registered student in a sortable table. Each row shows the student's name (with avatar initials), age, course badge, date added, and action buttons (Edit and Delete). The live search bar filters records instantly as the user types.

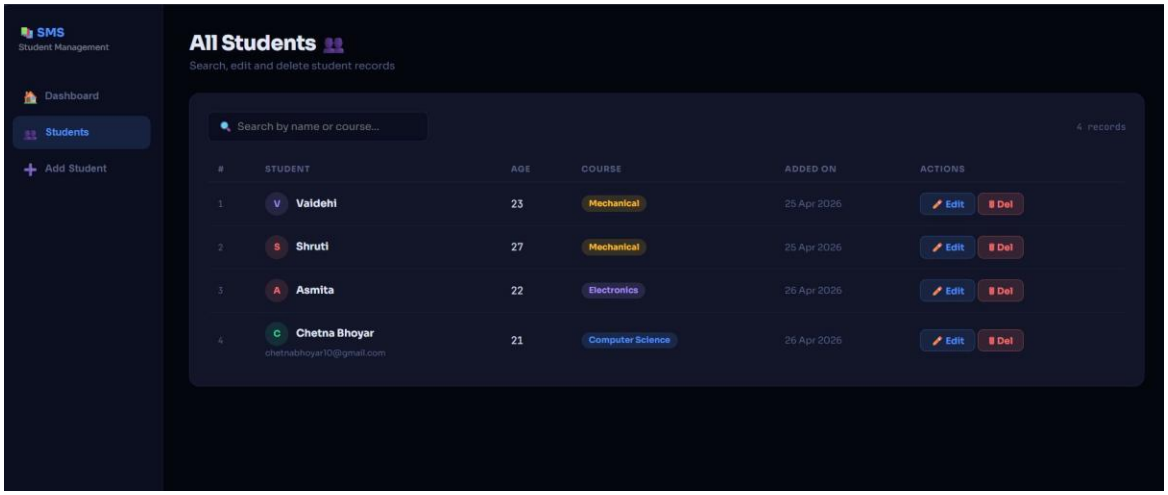

Figure 7.2 – All Students Page: Student Table with Search, Edit, and Delete

7.3 Web Application – Add Student Page

The Add Student page provides a registration form with fields for Full Name, Age, Course (dropdown), and optional Email. The 'What's New' panel on the right highlights key features. Clicking 'Add Student' validates the input and saves the record to localStorage, immediately reflecting on the Dashboard.

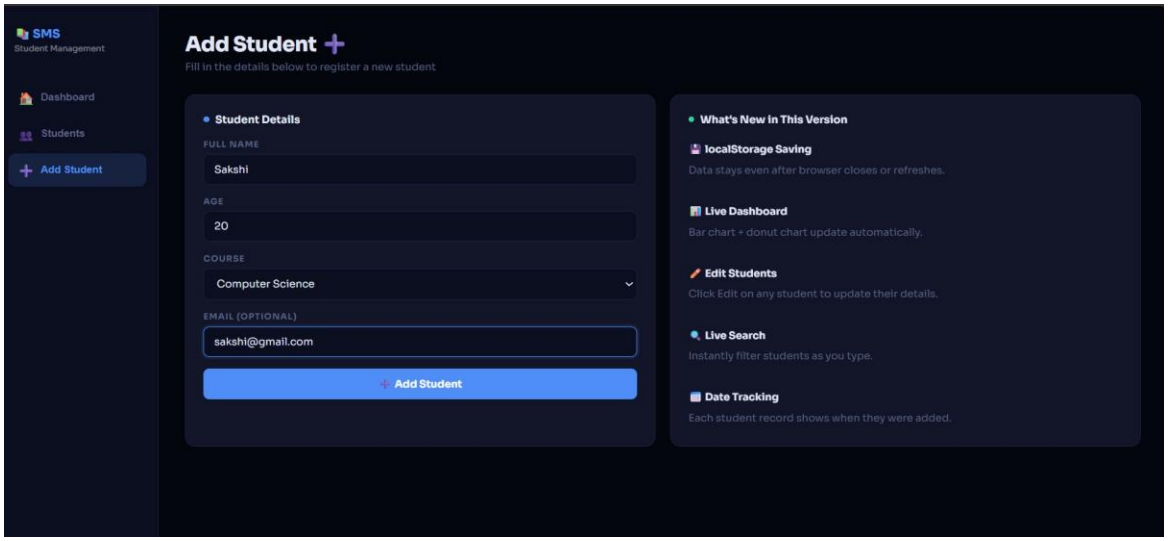

Figure 7.3 – Add Student Page: Registration Form with Validation

7.4 localStorage Data Persistence (Browser DevTools)

This screenshot from Chrome DevTools shows the Application → Local Storage panel. Under the key 'sms_students', the full JSON array of student records is stored. This demonstrates that data persists even after the browser is refreshed or closed, without requiring any server-side database.

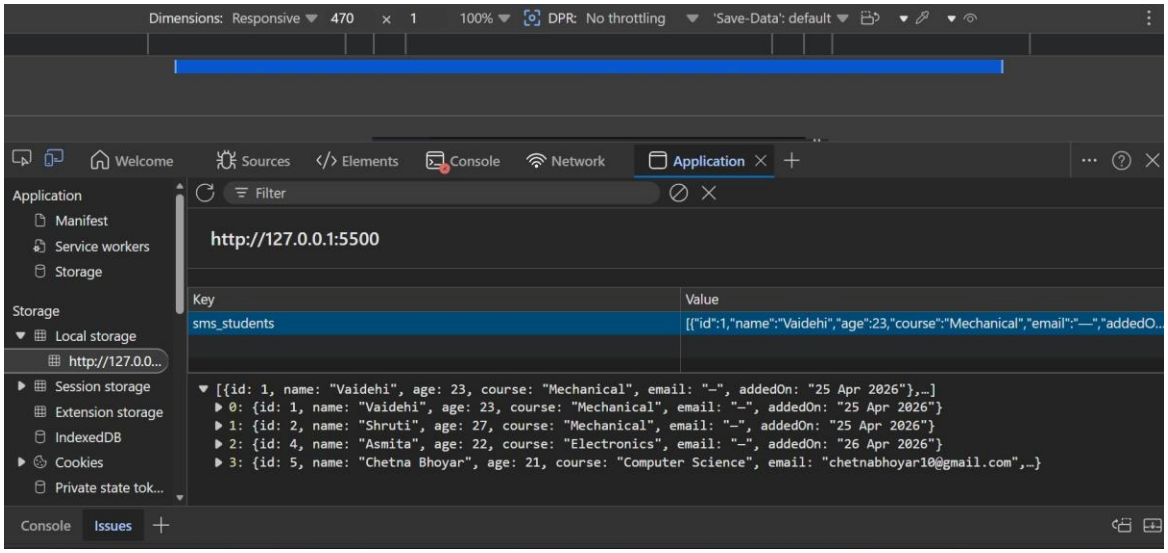

Figure 7.4 – Chrome DevTools: localStorage showing 'sms_students' JSON data

7.5 Java Console Application – Program Execution

The following screenshots show the Java console application running in the terminal. The program was compiled using 'javac' and executed with 'java StudentManagement'. The menu allows adding students (name, age, course), viewing all records, and exiting cleanly.

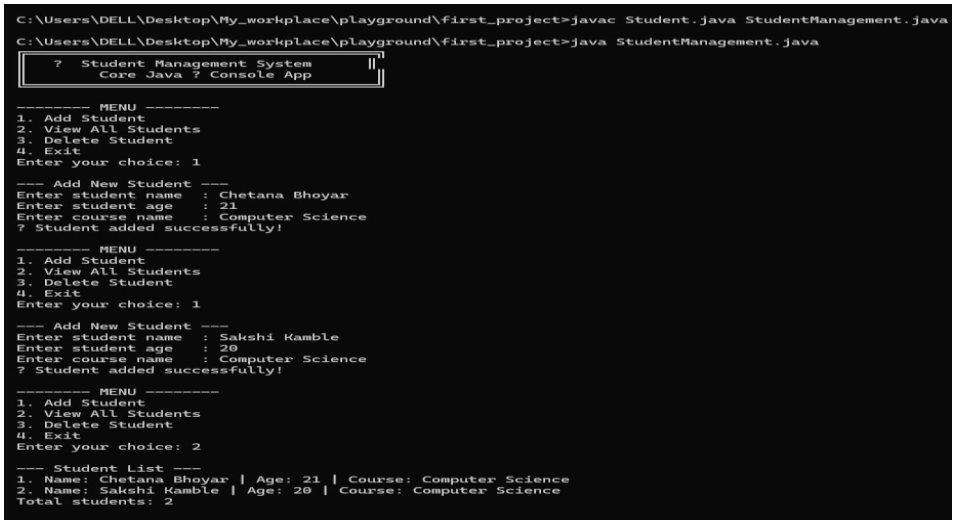

Figure 7.5 – Java Console Application: Compilation and Execution (Adding and Viewing Students)

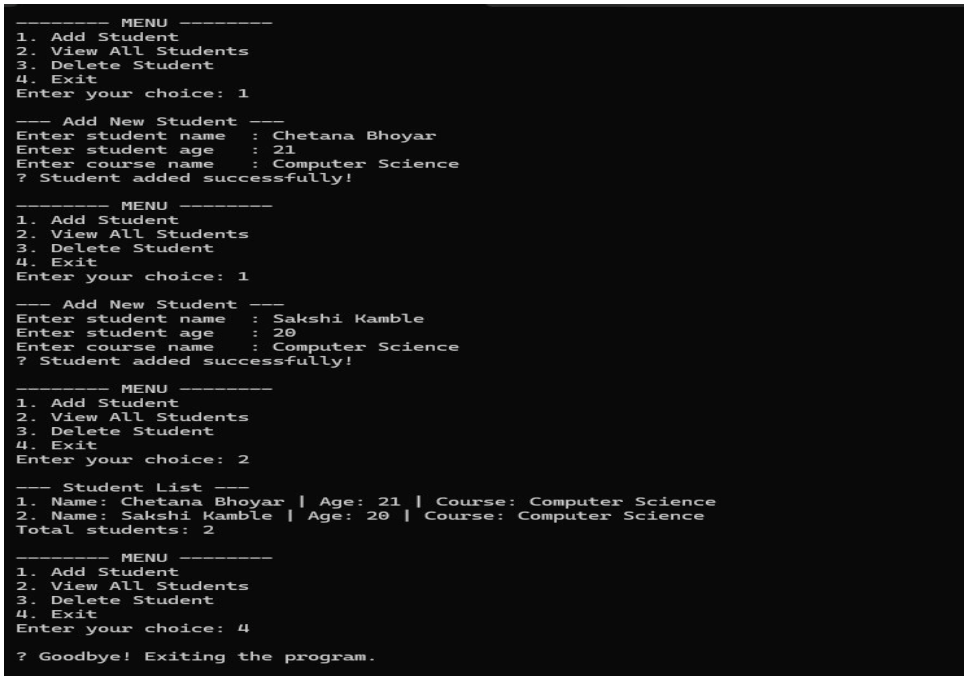

Figure 7.6 – Java Console: Menu Navigation, Add Student, View All, Exit

Chapter 8 – Testing

The system was tested using manual black-box testing, verifying all features against expected outputs for both valid and invalid inputs. Each test case was executed and results compared to expected behaviour.

Test ID	Test Case	Expected Output	Module	Result
TC-01	Add student with valid data	Student added and displayed in list	Java + Web	✓ Pass
TC-02	Add student with empty name	Error: Name cannot be empty	Java + Web	✓ Pass
TC-03	Add student with age = 10	Error: Age must be 16–60	Java + Web	✓ Pass
TC-04	Add duplicate student	Error: Duplicate record detected	Java	✓ Pass
TC-05	Search by partial name "ali"	Returns Alice and Salim	Java	✓ Pass
TC-06	Update student — press ENTER to skip field	Skipped field unchanged, others updated	Java	✓ Pass
TC-07	Delete with confirmation "no"	Deletion cancelled	Java	✓ Pass
TC-08	Delete with confirmation "yes"	Record removed from list	Java	✓ Pass
TC-09	Enter non-numeric menu choice	Error: Please enter a valid number	Java	✓ Pass
TC-10	Web: Add student, then refresh page	Data retained from localStorage	Web	✓ Pass
TC-11	Web: Live search by partial name	Table filters instantly as user types	Web	✓ Pass
TC-12	Web: Dashboard stat cards update	All stats reflect new record immediately	Web	✓ Pass

Chapter 9 – Limitations & Future Scope

9.1 Current Limitations

- Data is stored only in memory in the Java version — all records are lost when the program exits.
- No permanent database integration in the current version.
- The web version stores data in localStorage, which is browser-specific and not shared across devices.
- No user authentication or role-based access control (admin vs. student).
- The system currently supports a fixed set of courses with no ability to add custom courses.
- No attendance tracking, grade management, or performance analytics in the current version.

9.2 Future Scope

1. Integrate a Spring Boot REST API backend to serve data over HTTP, making the web frontend and Java backend truly connected.
2. Add a MySQL or PostgreSQL database with Spring Data JPA for permanent data persistence and multi-user support.
3. Implement user authentication with login/logout and role-based access control (Admin, Teacher, Student roles).
4. Add grade tracking, attendance management, and detailed performance analytics per student.
5. Build a mobile application version using React Native or Flutter for cross-platform support.
6. Deploy the web application to a cloud platform such as Vercel, Railway, or AWS for public access.
7. Add PDF export functionality for student reports, mark sheets, and attendance summaries.
8. Implement email notifications for student registration confirmation and important announcements.

Chapter 10 – Conclusion

The Student Management System successfully achieves its objective of providing a complete, functional, and well-structured application for managing student records in an educational setting. The project demonstrates a wide range of software development concepts, including Object-Oriented Programming, 3-layer architecture, input validation, data persistence, and visual data representation.

The Core Java console version effectively demonstrates academic programming principles such as encapsulation, constructors, Optional, and the Repository design pattern — making it a strong academic submission for a B.Com. Computer Application course.

The web-based version extends the project into the domain of modern web development, with a professional dark-themed dashboard, live charts, real-time search, and persistent storage using localStorage — demonstrating practical, deployable software skills.

Together, both versions represent a comprehensive project that bridges foundational Java programming with real-world web application development. The project forms a solid foundation for future enhancement with a Spring Boot backend, MySQL database, and user authentication — making it both academically rigorous and industry-relevant.

Chapter 11 – References

[1] Schildt, H. (2022). Java: The Complete Reference, 12th Edition. McGraw-Hill Education.

[2] Eckel, B. (2006). Thinking in Java, 4th Edition. Prentice Hall.

[3] Oracle Corporation. (2024). Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[4] Mozilla Developer Network. (2024). HTML, CSS & JavaScript Reference. https://developer.mozilla.org/

[5] W3Schools. (2024). Java Tutorial & Web Technologies. https://www.w3schools.com/

[6] GeeksforGeeks. (2024). Java Collections – ArrayList. https://www.geeksforgeeks.org/arraylist-in-java/

[7] Baeldung. (2024). Guide to Optional in Java. https://www.baeldung.com/java-optional

[8] Visual Studio Code. (2024). Java in VS Code. https://code.visualstudio.com/docs/languages/java